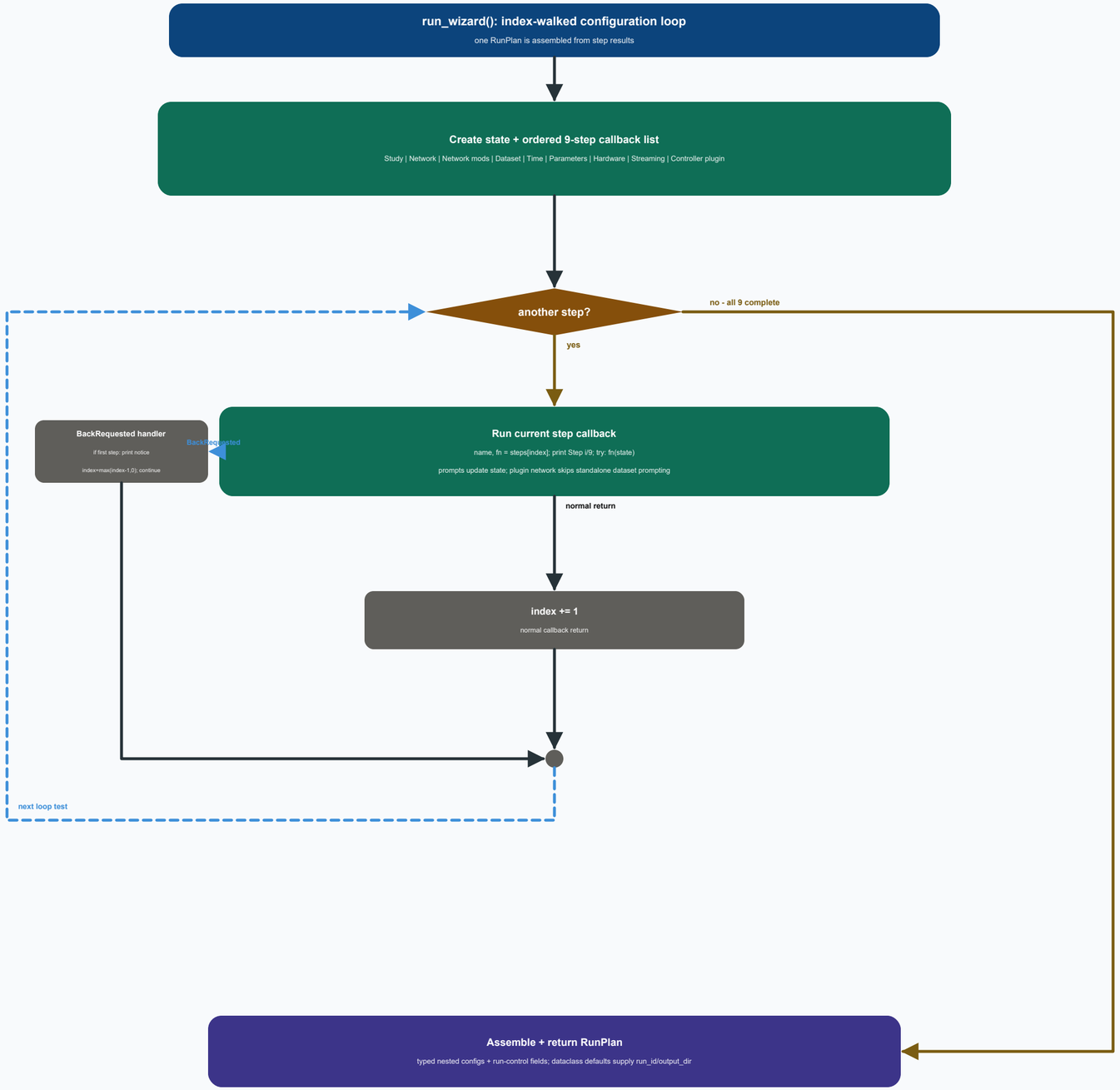

Nine configuration steps

Study chooses scenario comparison, voltage variation, hosting capacity or OPF; hosting capacity may additionally request the stressed re-benchmark.

Network supports curated presets, assembled SimBench codes, custom Python factories and network-plugin YAML.

Network modifications optionally inject PV sgens and flip selected switches before profile construction.

Dataset is SimBench native, DWD or custom for non-plugin networks; a network plugin supplies its own profile strategy and therefore skips this prompt.

The remaining steps select time window, limits/scaling/Q(V), dry-run or hardware mode, live-stream cadence and an optional controller plugin.

Navigation and validation

Menus expose 0 as Back; free-text prompts use '<'. Both raise BackRequested to the central loop.

BackRequested is caught around the current callback; the index is decremented with a floor at zero and the while-loop is re-tested.

At index 0 the handler prints an already-at-first-step notice, then re-enters Step 1; successful callbacks increment the index.

Prompt-level validation catches ranges and input syntax here; cross-field compatibility and filesystem/runtime checks remain the executor's responsibility.

RunPlan data model

RunPlan nests NetworkConfig, DatasetConfig and ParameterConfig and adds study, hardware, controller-plugin, HC, time-window and output fields.

The wizard supplies the selected configuration fields; dataclass defaults create the fresh run_id and default output_dir for a newly assembled RunPlan.

to_dict()/from_dict() provide the JSON serialization boundary used by saved presets and the executor's reproducibility copy.

The data classes are pure configuration containers; runtime decisions remain in wizard.py and executor.py.